

複雑なデータモデリングにおけるプログラミング言語の「区間(Range)」構文と境界意味論の網羅的分析

1. 序論: 区間の数学的性質とデータ型の境界がもたらす複雑性

プログラミングにおいて「範囲(Range)」または「区間(Interval)」を表現する機能は、配列のスライス操作、データベースのクエリ構築、反復処理(ループ)、さらには高度な科学技術計算に至るまで、あらゆる場面で利用される中核的な概念である。しかし、この一見単純な概念は、対象となるデータ型が「離散的(Discrete)」であるか「連続的(Continuous)」であるかによって、その本質的な複雑さを露呈する。

ユーザーのクエリで指摘されている通り、整数(Integer)のように隣接する値が明確に定義されている離散型データの場合、閉区間 $[1, 5]$ と半开区間 $[1, 6)$ は数学的に等価な要素の集合を表現

する。そのため、言語の仕様としてどちらか一方が採用されていても、必要に応じて上限値に 1 を加算・減算することで、プログラマは構文の差異を吸収することが可能である。PHPの `range(1, 5)` が閉区間として振る舞う一方で、Pythonの `range(1, 5)` が半开区間として振る舞うという違いは、離散データの世界においては単なるオフセットの調整に過ぎない。

しかし、浮動小数点数(Float)や日時(Datetime)のような連続型データへと型を拡張した瞬間、この前提は完全に崩壊する。区間 $[1.0, 5.0]$ と $[1.0, 5.0)$ の間には無限の要素が存在し、「5.0の直前の値」をプログラミング言語上で $+1$ や -1 のような単純な定数の加減算で指定することは

不可能である¹。結果として、言語やライブラリ、静的解析ツールは、开区間 (a, b) 、閉区間 $[a, b]$

、そして半开区間 $[a, b)$ または $(a, b]$ を構文レベル、あるいは型レベルで厳密に書き分ける手段を提供しなければならない。

本稿では、コンピュータサイエンスにおける半开区間の歴史的背景から出発し、現代の主要なプログラミング言語(Rust, Swift, Kotlin, Python, C#, Julia, C++など)、静的解析ツール(PHPStan, TypeScriptエコシステム)、データベース(PostgreSQL)、さらには高精度な区間演算の国際標準規格(IEEE 1788)に至るまで、各技術がこの「連続と離散の境界問題」をどのように解決しているかを網羅的かつ深層的に分析する。

2. 理論的基盤: Dijkstraの半开区間原則と離散データの処理

現代の多くのプログラミング言語が、配列のインデックスや反復処理においてデフォルトで半开区間 $[a, b)$ を採用している背景には、1982年に計算機科学者 Edsger W. Dijkstra が執筆した草稿「

Why numbering should start at zero (EWD831)」の強い影響が存在する³。Dijkstraは、自然数の部分列を表現するための4つの数学的記法を比較検討し、プログラムのロジックにおいてどの記法が

最もバグを生みにくいかを論じた。

Dijkstraが検討した4つの記法は以下の通りである。第一に $a \leq i < b$ (左閉右開の半開区間)、第二に $a < i \leq b$ (左開右閉の半開区間)、第三に $a \leq i \leq b$ (閉区間)、第四に $a < i < b$ (开区間)である⁴。彼は、閉区間を用いると、空の区間(要素を持たないシーケンス)を

表現する際に、上限が下限を下回るという不自然な状態(例: $0 \leq i \leq -1$)を生み出すと指摘した⁴。また、下限を開区間にする、最小の自然数(例えばゼロ)から始まるシーケンスを指定する際に、下限値として不自然な負の数(例: $-1 < i < 3$)を使わざるを得なくなる⁴。

結果として、下限を含み上限を含まない $a \leq i < b$ の半開区間記法が最も優れていると結論付けられた。この半開区間の性質は、プログラミングにおいてフェンスポスト問題(Off-by-one error)

を回避し、論理的な一貫性を保つ上で極めて有効である³。区間の長さは単純な引き算 $b - a$ で求まり、区間 $[a, c)$ を任意の点 b で分割した場合も $[a, b)$ と $[b, c)$ に綺麗に分かれ、境界値の重複や欠落が発生しない³。さらに、開始と終了が等しい $[a, a)$ は長さゼロの空区間として直感的に扱うことができる³。

C言語の標準的な for (int i = 0; i < N; i++) ループや、C#の範囲演算子、Pythonのスライス構文などは、すべてこの半開区間の哲学とゼロオリジンインデックスの組み合わせによって構築されている⁶。しかしながら、これらの設計はあくまで「インデックス(オフセット)」としての整数演算を前提としたものであり、連続的なデータを扱う場合には、閉区間や开区間を明示的に表現する新たな構文の必要性が生じることになる。

3. 離散型データにおける区間表現とスライスの実態

離散データ(主に配列や文字列のインデックス)を操作するための機能として、各言語は独自の構文やオブジェクトを提供している。これらの実装を詳細に観察すると、半開区間がいかにしてメモリ上のデータ構造の操作に最適化されているかが理解できる。

3.1. Pythonにおけるスライスと不連続な意味論

Pythonは range(start, stop) 関数や slice オブジェクト、そして a[start:stop:step] という構文を通じて、半開区間を強力にサポートしている⁸。Pythonのスライス機能は、Dijkstraの原則に忠実であり、指定された stop インデックスは決して結果に含まれない(Exclusive)⁸。

しかし、Pythonのスライス構文は利便性を追求した結果、内部的に非常に複雑な境界論理を抱えている。Quansight Labsの解析によれば、Pythonのスライスは引数の値(正、負、省略)によってその数学的な意味合いが不連続に変化する¹⁰。例えば、負のインデックスを使用すると末尾からのオフセットとなり(-1が最後の要素)、ステップ値が負の場合はイテレーションの方向が逆転する。このとき、区間は依然として半開区間であるが、「境界を含まない」対象が右側から左側に切り替わる¹⁰。C言語のAPIレベル(PySlice_GetIndicesEx 等)で見ると、これらの範囲外インデックスはシーケンスの長さに合わせて自動的にクリッピングされ、エラーを投げることなく空のリストを返すように調整されている¹⁰。これは、要素の探索や抽出において極めて堅牢であるが、数学的な区間表現としては純

粋さを欠く側面もある。

3.2. C#とChapelにおける専用構文

C# では、配列やコレクションのスライスを簡潔に記述するため、..¹³ という範囲演算子 (Range operator) が導入されている。これはPythonと同様に半开区間を生成し、ReadOnlySlice などのメモリ効率の高いデータ構造と組み合わせて、文字列の部分抽出などで新たなオブジェクトのメモリアロケーションを避ける用途に多用される¹⁴。

また、並行計算言語である Chapel でも、..¹³ は区間を生成する演算子として機能するが、さらに明確に..¹³ 演算子が半开区間を定義するものとして提供されている。Chapel では、区間同士の代入において暗黙の型変換が許容される場合、ストライド (step) やアライメントも引き継がれる高度な機能を持つ¹³。

3.3. Crystalにおける閉区間と半开区間の共存

Rubyの影響を強く受ける静的型付け言語 Crystal では、文字列や配列のスライス操作において、閉区間と半开区間をドットの数で視覚的に区別する構文を採用している¹⁵。

構文 (Crystal)	意味論	文字列スライスの例 (s = "Hello")
a..b	閉区間 (bを含む)	s[0..1] → "He"
a...b	半开区間 (bを含まない)	s[0...1] → "H"

このように、離散型のインデックス操作を中心とするコンテキストであっても、ドットの数による書き分けは、オフセット計算の記述を直感的にする上で一定の価値を提供している。しかし、これが浮動小数点数に適用されると、意味論的な重要性は飛躍的に高まる。

4. 連続型データの台頭と専用構文の進化

整数のループや配列インデックスの用途を超え、浮動小数点数、日付、あるいは文字列の辞書順比較といった「連続型データ (あるいはそれに準ずるデータ)」を区間として扱う必要性が高まると、「上限から1を引く」という離散型の妥協案は通用しなくなる。モダンなプログラミング言語は、この問題に対処するために区間演算子の構文を再設計してきた。

4.1. Kotlin: until の限界と ..< 演算子の導入

Kotlinの区間構文の進化は、離散型から連続型への移行によるパラダイムシフトを見事に体現している。初期のKotlinでは、閉区間には a..b (rangeTo 関数) を用い、半开区間には a until b という中置関数を用いていた²。

しかし、until には致命的な欠陥があった。この関数は整数型 (Int, Long など) に対してのみ定義されており、内部実装としては単純に a..(b - 1) という閉区間に変換して処理されていたのである²。これはまさに「整数なら上限から1を引けば済む」という離散的アプローチである。この手法は、文字列 (例: "cat" と "dog" の間の単語) や浮動小数点数といった連続型データには適用できない。"dog" から 1 を引く演算は定義されていないためである²。

この問題を解決するため、Kotlin 1.9以降では、新たに半開区間演算子 `..<` (`rangeUntil` 関数) が導入された²。これにより、内部的な減算変換を伴わずに、任意の `Comparable` 型に対して真の半開区間 (`OpenRange`) を構築できるようになった。これにより、浮動小数点数に対しても `0.0..<5.0` のような表現が可能となり、連続値の包含判定が正確に行えるようになった²。

4.2. Swift: 直感的なドット構文と汎用性

Swiftは当初から、区間を表現するために視覚的に明確な2つの演算子を提供し、それを言語のコア設計に組み込んでいる¹⁸。

演算子 (Swift)	名称	数学的意味	適用対象
<code>a...b</code>	閉区間演算子 (Closed Range Operator)	$[a, b]$	任意の <code>Comparable</code> 型
<code>a..<b</code>	半開区間演算子 (Half-Open Range Operator)	$[a, b)$	任意の <code>Comparable</code> 型

Swiftの優れた点は、これらの区間が最初から `Comparable` プロトコルに準拠する任意の型を対象としていることである。`let underFive = 0.0..<5.0` という浮動小数点数の区間を作成し、`underFive.contains(3.14)` を評価すれば `true` が返り、`contains(5.0)` であれば厳密に `false` が返る¹⁹。Swiftの `switch` 文におけるパターンマッチングでも、この区間演算子はそのま利用でき、スコアの判定 (例: `80..<90`) などにおいて境界値の曖昧さを完全に排除している²⁰。

4.3. Rust: パターンマッチングと型の安全性

Rustも同様に、2種類の演算子を用意しているが、構文の視覚的な曖昧さを避けるための歴史的な変更を経ている⁵。

初期のRustでは、閉区間に `...` が使用されていたが、半開区間の `..<` と視覚的に見間違えやすい(ドットを数え間違えることによる致命的な論理バグを誘発する) という理由から非推奨となった。代わりに、等号を含んで包括性 (Inclusive) を明示する `..=` が導入された²¹。

- 半開区間: `a..b` (`std::ops::Range`)
- 閉区間: `a..=b` (`std::ops::RangeInclusive`)

Rustにおける区間の興味深い課題は、浮動小数点数 (`f32`, `f64`) のパターンマッチングにおいて現れる。これについては後述のセクションで深く掘り下げる。

5. 静的解析と型システムにおける区間の表現

実行時の値としてではなく、「型」そのものとして区間を定義し、コンパイル時や静的解析時にプログラムの安全性を証明するアプローチも進化を遂げている。

5.1. PHPStan と Psalm: 離散型の厳密な境界定義

PHPの強力な静的解析ツールである PHPStan や Psalm では、ジェネリクス構文を応用して、整数の区間を独自の型として表現する機能が提供されている²²。

Psalmの型システムにおいては、以下のような区間型が定義可能である²³。

- `int<1, 5>` : 1から5までの整数(閉区間)
- `int<0, max>` : 0以上の整数(non-negative-int と等価)
- `int<min, -1>` : 負の整数(negative-int と等価)

これらの型システムは、離散的な整数に対して極めて強力に機能し、配列のインデックス外アクセス、ゼロ除算、あるいはビットマスク(`int-mask<1, 2, 4>`)の検証に寄与する²³。

しかし、ユーザーのクエリが指摘するように、浮動小数点数(Float)に対して `float<1.0, 5.0>` のような区間型を静的解析ツールがネイティブで提供することは稀である。これは、浮動小数点数が連続空間であり、無限の状態空間を持つこと、さらに演算のたびに丸め誤差が生じるため、コンパイル時に変数の値が厳密に区間内に収まるかを演繹的に証明することが計算量的に困難(あるいは不可能)であることに起因していると考えられる²²。Psalm等の型システムでは、float はあくまでスカラー型のスーパータイプの一部として扱われ、細かい値の区間制約はサポートされていない²³。

5.2. TypeScript と実行時スキーマバリデーション

TypeScriptでも同様に、数値の範囲をネイティブな型レベルで制約する機能(例えば `number & Minimum & Maximum`)は長らく要望されているものの、実装には至っていない。現在提案されている `type Range = [start: number, end: number]` といった機能も、タプル型の要素数の制約に関連するものであり、連続値の値の範囲を規定するものではない²⁶。

この静的型付けの限界を補うため、TypeScriptエコシステムでは ArkType, Zod, Valibot といった実行時バリデーションライブラリが利用される²⁸。これらは、データベースのクエリビルダーと同様のアプローチを採用し、明示的な論理比較演算子のエイリアスを使用することで开区間と閉区間を書き分ける²⁸。

- `gt (Greater Than):` $> a$ (开区間の下限)
- `gte (Greater Than or Equal):` $\geq a$ (閉区間の下限)
- `lt (Less Than):` $< b$ (开区間の上限)
- `lte (Less Than or Equal):` $\leq b$ (閉区間の上限)

例えば、`[1.0, 5.0]` という半开区間をスキーマとして定義する場合、`{ gt: 1.0, lte: 5.0 }` のようなオブジェクト表現を用いる。これは、連続値の区間表現において、無限の要素を持つ区間の境界を厳密に定義し、型推論と実行時検証のギャップを埋めるための現実的かつ堅牢なアプローチである²⁸。

6. 浮動小数点数と区間演算の深淵: IEEE 754の制約

浮動小数点数を区間として扱う場合、単に「境界の開閉」を指定するだけでは不十分であり、基礎となる IEEE 754 規格に起因する深刻な異常状態(Anomalies)を考慮しなければならない。

6.1. NaNによる順序付けの崩壊

Rustの浮動小数点数(`f32`, `f64`)のドキュメントが明記している通り、浮動小数点数は完全な順序付け(Total Ordering)を持たない³¹。その最大の原因は NaN(Not a Number)の存在である。

NaN は自分自身とも等しくない(`NaN == NaN` は常に `false` を返す)。さらに、いかなる浮動小数点数と比較しても、`NaN < 1.0` や `NaN > 1.0` はすべて `false` となる³¹。これにより、Rustの `f32` 型は同値関係を示す `Eq` トレイトや、完全順序を示す `Ord` トレイトを実装していない³²。

この数学的特性は、パターンマッチングや区間表現において致命的な問題を引き起こす。例えば、浮動小数点数の区間 $0.0..5.0$ に対して `contains(NaN)` を評価した場合の論理的帰結はどうか。比較演算子がすべて `false` を返すため、境界チェックは予測不可能な挙動を示す可能性がある³¹。また、負のゼロ (-0.0) と正のゼロ ($+0.0$) は比較上は等価 (`==`) とされるが、除算などの算術演算

を経由すると異なる符号の無限大 ($-\infty$, $+\infty$) を生成するため、区間の境界として扱った場合に連続性が破綻するケースがある³²。

6.2. qNaNとsNaNのペイロード

さらに、IEEE 754規格では、NaNのビットパターンに「ペイロード」と「シグナルビット」を含めることを許容している。Rustの実装では、シグナルビットが1のものを Quiet NaN (qNaN)、0のものを Signaling NaN (sNaN) と解釈する³²。これらは算術演算の過程で非決定的なビットパターンを生成する可能性があり、コンパイル時の定数評価 (const コンテキスト) と実行時の評価で異なる NaN が生成されることすらある³²。

このようなIEEE 754固有の複雑な非決定性が存在するため、単純な「浮動小数点数の区間 (`float<1.0, 5.0>`)」を型システム上で完全な静的証明として組み込むことは極めて困難であり、多くの言語がこれを断念している背景となっている。

7. 高度な数学的モデリングと標準規格 (IEEE 1788)

標準の構文 (`..` や `..<`) では表現しきれない複雑な区間操作や、浮動小数点数の誤差を数学的に厳密に扱うため、多くの言語では高度なサードパーティライブラリや標準ライブラリ、さらには国際標準規格に基づくアプローチが提供されている。

7.1. Haskell: Allenの区間代数と `Ix` クラス

純粋関数型言語である Haskell では、`Ix` クラスを用いて連続的な部分範囲を整数にマッピングする手法が取られる³⁶。`inRange` 関数は境界内に値が存在するかを判定し、`rangeSize` 関数は区間のサイズを計算する³⁶。より高度な要件に対しては `data-interval` のようなライブラリが利用され、開区間と閉区間の両方をサポートすると同時に、区間同士の包括・交差・隣接関係を網羅する「Allenの区間代数 (Allen's Interval Algebra)」に基づく関係演算を実装している³⁷。これにより、2つの区間が論理的にどのように重なり合うかを数学的に証明・演繹することが可能となる³⁸。

7.2. C++ (Boost.ICL): 静的境界と連続空間のタイル化

C++の Interval Container Library (Boost.ICL) は、離散型と連続型の違いをライブラリのアーキテクチャの根幹で意識して設計されている⁴⁰。

Boost.ICLは、区間の境界を静的な型パラメータとして表現する。

- `right_open_interval<T>`: $[a, b)$
- `left_open_interval<T>`: $(a, b]$
- `closed_interval<T>`: $[a, b]$
- `open_interval<T>`: (a, b)

ここで特筆すべきは、データ型に応じて推奨されるデフォルトの区間が異なる点である。整数や日付のような離散型データに対しては `discrete_interval` が用いられる一方で、実数や文字列などの連続型データには `continuous_interval` が利用される⁴¹。連続型データにおいて半开区間 (`..<` や `<..`)

right_open_interval) がデフォルトとして扱われるのは、連続空間をオーバーラップも欠落もなく分割・結合(タイル化)する際に、数学的特性が最も優れているためである⁴¹。

7.3. Julia: 数学的記法との完全な統合

科学技術計算を主眼とする Julia では、区間演算に対する要求が極めて高い。Julia の IntervalSets.jl パッケージは、数学表記とプログラミングの親和性を極限まで高めた設計を持つ⁴⁴。通常の演算子として $1.0 \dots 3.0$ や 1.5 ± 1 を使って閉区間を表現できるほか、Interval{open, :closed}(1, 3) といった型パラメータを用いた厳密な定義が可能である⁴⁵。さらに、Julia 特有の強力な文字列マクロ機能を利用し、数学的なブラケット表記をそのままコードに記述できる機能(@iv_str)を提供している⁴⁵。

- `iv"[1, 5]"` → 閉区間 $[1, 5]$
- `iv"[1, 5)"` → 半開区間 $[1, 5)$
- `iv"(1, 5]"` → 半開区間 $(1, 5]$
- `iv"(1, 5)"` → 开区間 $(1, 5)$

これは、構文レベルで解析処理をフックできる Julia の特性を活かし、浮動小数点数に対する無限の要素数を持つ区間表現を、数学者のメンタルモデルと完全に一致させた秀逸な設計である⁴⁵。

7.4. IEEE 1788 区間演算標準と装飾 (Decorations)

区間同士の四則演算(区間演算)の精度と安全性を国際的に標準化したのが、IEEE 1788 (Standard for Interval Arithmetic) である⁴⁷。

浮動小数点数には丸め誤差が存在するため、例えば区間 $[a, b] + [c, d]$ を計算した結果の境界値が、真の数学的な境界値をわずかに下回る(または上回る)ことがあってはならない。「Thou shalt not lie (汝、嘘をつくことなかれ)」という原則に従い、IEEE 1788 では、区間演算の結果は常に真の解を完全に包含するよう外側への丸め(Outward rounding)を行うことが規定されている⁴⁹。

さらに、IEEE 1788 において極めて重要なのが、装飾(Decorations)という概念である⁴⁸。これは区間に付与されるメタデータであり、その区間がどのような演算を経て生成されたか、関数とその区間上で連続であったか等の状態を追跡する。

装飾 (Decoration)	意味・状態
COM (Common)	演算が完全に有効であり、区間が空ではなく、関数が連続であったことを保証する。
DAC (Defined and Continuous)	区間は無制限(Unbounded)かもしれないが、関数は連続である。
DEF (Defined)	値は定義されているが、不連続性が存在する可能性がある。

TRV (Trivial)	単一の点、あるいは空の区間などの自明な状態。
ILL (Illegal)	無効な演算によって生じた不正な区間 (Nai: Not an Interval)。

浮動小数点数における区間では、例えば $1/[-1, 2]$ のような「ゼロを跨ぐ区間での除算」を行った場合、単純な閉区間では表現できず、本来は $[-\infty, -1] \cup [0.5, \infty]$ のように2つの分離した区間（あるいは無制限の区間）となる⁴⁸。IEEE 1788準拠のライブラリ (Juliaの IntervalArithmetic.jl など) は、このような極限状態においてもシステムが破綻しないよう、装飾システムを用いて安全なフォールバックと追跡可能性を担保している⁵³。

8. データベースにおける永続化と「正規化 (Canonicalization)」のメカニズム

プログラミング言語上のメモリ内データ構造としてだけでなく、データを永続化するデータベース (RDBMS) においても、区間の扱いは極めて重要である。PostgreSQLは、ネイティブで強力な区間型 (Range Types) をサポートしており、離散型と連続型の区別の問題を「正規化 (Canonicalization)」という概念でエレガントに解決している¹。

PostgreSQLは以下の主要な区間型を提供する⁵⁴。

区間型	データ型	特性
int4range, int8range	整数	離散型 (Discrete) ⁵⁴
daterange	日付	離散型 (Discrete) ⁵⁴
numrange	数値 (Numeric)	連続型 (Continuous) ⁵⁴
tsrange, tstzrange	タイムスタンプ	連続型 (Continuous) ⁵⁴

コンストラクタや文字列表現として、数学と同様に `[]` と `()` を使用して境界の開閉を柔軟に指定することができる (例: `'[3, 7)'`、あるいは関数を用いて `numrange(1.0, 14.0, '()')`)⁵⁴。ここでデータベースエンジンにおける最も重要な機能が、離散型データに対する正規化関数 (Canonical function) の自動適用である¹。例えば、整数型区間 `int4range` に対して閉区間 `'[1, 10]'` を挿入した場合、PostgreSQLはこれが「明確なステップ幅を持つ離散型」であることを認識し、自動的に標準形式である半开区間 `'[1, 11)'` に変換 (正規化) してストレージに保存する¹。これにより、内部的なB-TreeインデックスやGiSTインデックスを用いた比較演算において、表現の揺れ (`[1, 10]` と `[1, 11)` が混在すること) がなくなり、高速かつ正確な包含 (`@>`) やオーバーラップ (`&&`) の判定が可能とな

る¹。

一方で、浮動小数点数や高精度数値の `numrange` の場合、要素間に「次の値」という概念が存在しない連続空間であるため、この正規化は決して行われない¹。'[1.0, 10.0]' はそのまま '[1.0, 10.0]' として保存され、システムが勝手に上限値を操作することはない⁵⁹。また、前述した「ゼロ除算による区間の分割」のような問題に対処するため、PostgreSQLは `multirange` 型(順序付けられた複数の非連続区間のリスト)も提供しており、複雑な集合演算をネイティブにサポートしている⁵⁴。

この挙動の差異は、ユーザーのクエリで提起された「整数なら $+1$ で書き換えられるが、浮動小数点数ではそうはいかない」という根本的な命題を、データベースシステムが型アーキテクチャのレベルで深く理解し、型ごとに内部表現を分岐させて実装している完璧な実例である¹。

9. 結論

本調査により、プログラミング言語および関連ツールにおける区間(Range/Interval)の構文と機能は、対象とするデータが「離散的(Discrete)」であるか「連続的(Continuous)」であるかによって、設計思想が明確に二分され、それぞれ異なる進化を遂げてきたことが明らかになった。

1. 離散データの妥協とDijkstraの遺産: 整数のみを対象とした時代やコンテキスト(配列のインデックス、スライス等)においては、Dijkstraが提唱した「半开区間 $[a, b)$ が至高である」という思想に基づき、シンプルな反復構文が標準となった。Pythonのスライスや、Kotlinの旧 `until` メソッド、PostgreSQLの離散型正規化が示すように、離散データにおいては「閉区間を半开区間に変換する($+1$ する)」という内部処理によって、複雑な構文を増やさずに処理を一本化することが可能であった。
2. 連続データによる構文のパラダイムシフト: 浮動小数点数や日時データに対する処理の要求が高まるにつれ、無限の要素を持つ区間の境界を正確かつ直感的に指定する必要性が生まれた。これにより、Swift の `...` と `..<`、Rust の `..=` と `..<`、そして Kotlin における `until` から `..<` への進化が示すように、コンパイラレベルで境界の開閉を視覚的に区別する専用の演算子構文が各言語に次々と導入された。
3. 型システムとデータベースの適応: PHPStan のような静的解析は整数の限界内に留まっているが、ZodやArkTypeなどのTypeScriptランタイムバリデーターや、PostgreSQLのようなデータベース層は、`gt`、`lte` のような論理演算子のマッピングや、`numrange` のように正規化をバイパスする専用の型定義によって、連続空間の無限性を安全にカプセル化している。
4. 究極の抽象化と数学的厳密性: C++ の `Boost.Interval` や Julia の `IntervalSets.jl`、そして IEEE 1788 規格は、开区間・閉区間・半开区間のあらゆる組み合わせを型やマクロとして提供し、浮動小数点特有の丸め誤差や関数の連続性(Decorations)までも追跡する。ここでは、プログラミング言語の構文が、限界を伴う計算機科学の枠を超えて、純粋な数学的記法へと完全に統合されている。

開発者および言語設計者は、対象となるドメインが整数のインデックス処理に留まるのか、あるいは連続的な実数空間を扱うのかを深く理解した上で、適切な言語機能、ライブラリ、およびデータベースの型設計を選択する必要がある。区間の境界における開閉の制御は、もはや単なる「1のズレ」や「オフセット」を防ぐための便宜的な機能ではなく、無限の連続空間をコンピュータ上で安全かつ厳密にモデリングするための、不可欠な数学的・構造的基盤である。

引用文献

1. This is the effect of canonicalization in PostgreSQL - HEY World,
<https://world.hey.com/dinom/this-is-the-effect-of-canonicalization-in-postgresql-3ed5507c>
2. what is the difference between kotlin ..< operator and until operation for ranges,
<https://stackoverflow.com/questions/76961954/what-is-the-difference-between-kotlin-operator-and-until-operation-for-range>
3. Why do programmers prefer half-open intervals in Python, and what makes them easier to work with than inclusive intervals? - Quora,
<https://www.quora.com/Why-do-programmers-prefer-half-open-intervals-in-Python-and-what-makes-them-easier-to-work-with-than-inclusive-intervals>
4. Why Numbering Should Start At Zero - C2 Wiki,
<https://wiki.c2.com/?WhyNumberingShouldStartAtZero>
5. Range syntax is confusing - language design - Rust Internals,
<https://internals.rust-lang.org/t/range-syntax-is-confusing/10573>
6. What is half open range and off the end value - Stack Overflow,
<https://stackoverflow.com/questions/13066884/what-is-half-open-range-and-off-the-end-value>
7. Python slicing operator [start:stop:step] - Stack Overflow,
<https://stackoverflow.com/questions/75378378/python-slicing-operator-startstop-step>
8. slice() | Python's Built-in Functions,
<https://realpython.com/ref/builtin-functions/slice/>
9. Python Slice: Useful Methods for Everyday Coding - DataCamp,
<https://www.datacamp.com/tutorial/python-slice>
10. Slices - ndindex documentation - GitHub Pages,
<https://quansight-labs.github.io/ndindex/indexing-guide/slices.html>
11. Slicing in Python: A Comprehensive Guide - Towards Data Science,
<https://towardsdatascience.com/slicing-in-python-a-comprehensive-guide-a609c3bb877c/>
12. Slice Objects — Python 3.14.7 documentation,
<https://docs.python.org/3/c-api/slice.html>
13. Ranges — Chapel Documentation 2.9 - The Chapel Programming Language,
<https://chapel-lang.org/docs/language/spec/ranges.html>
14. Proposal: Slicing · Issue #120 · dotnet/roslyn - GitHub,
<https://github.com/dotnet/roslyn/issues/120>
15. String - Crystal 1.21.0 - The Crystal Programming Language,
<https://crystal-lang.org/api/latest/String.html>
16. Crystal Programming Language: The Ultimate Guide for Developers in 2026,
<https://mustafadevstudio.netlify.app/330-2/>
17. Ranges and progressions | Kotlin Documentation,
<https://kotlinlang.org/docs/ranges.html>
18. Basic Operators - Documentation | Swift.org,
<https://docs.swift.org/swift-book/documentation/the-swift-programming-language>

- [ge/basicoperators/](#)
19. Range | Apple Developer Documentation, <https://developer.apple.com/documentation/swift/range>
 20. Switch With Ranges – Swift – Caddy.tech, https://caddy.tech/learn/swift/fundamentals/switch_with_ranges
 21. Tracking issue for ``..X``, and ``..=X`` (``#![feature(half_open_range_patterns)]``) · Issue #67264 · rust-lang/rust - GitHub, <https://github.com/rust-lang/rust/issues/67264>
 22. Writing code for static analysis - Shopware Developer Documentation, <https://developer.shopware.com/docs/resources/guidelines/code/core/writing-code-for-static-analysis.html>
 23. Scalar types - Documentation - Psalm, https://psalm.dev/docs/annotating_code/type_syntax/scalar_types/
 24. PHPDocタイプ - BEAR.Sunday, <https://bearsunday.github.io/manuals/1.0/ja/types.html>
 25. How Psalm represents types - Documentation, https://psalm.dev/docs/running_psalms/plugins/plugins_type_system/
 26. Documentation - TypeScript 4.0, <https://www.typescriptlang.org/docs/handbook/release-notes/typescript-4-0.html>
 27. Announcing TypeScript 4.0 - Microsoft Developer Blogs, <https://devblogs.microsoft.com/typescript/announcing-typescript-4-0/>
 28. GitHub - productdevbook/unadapter: Universal, type-safe database adapter layer with a unified API for Drizzle, Prisma, Kysely, Knex, MongoDB, Sumak and in-memory backends, <https://github.com/productdevbook/unadapter>
 29. BrainDAO/prompt-weaver: A powerful, extensible template engine for building prompts - GitHub, <https://github.com/IQAIcom/prompt-weaver>
 30. GitHub - MunMunMiao/ck-orm: A typed ClickHouse query layer for modern JavaScript runtimes, with schema DSL, query builder, raw SQL escape hatches, session helpers, and observability hooks., <https://github.com/MunMunMiao/ck-orm>
 31. A Beginner's Guide to Mastering Data Types in Rust | by Uday Hiwarale | rustycrab | Medium, <https://medium.com/rustycrab/a-beginners-guide-to-mastering-data-types-in-rust-c784c79810b2>
 32. f32 - Rust Documentation, <https://doc.rust-lang.org/std/primitive.f32.html>
 33. f32 - kernel - Rust, <https://rust.docs.kernel.org/core/primitive.f32.html>
 34. Pattern Matching Limitations : r/rust - Reddit, https://www.reddit.com/r/rust/comments/wemrdz/pattern_matching_limitations/
 35. 3514-float-semantics - The Rust RFC Book, <https://rust-lang.github.io/rfcs/3514-float-semantics.html>
 36. Zvon - Haskell Reference, <http://www.zvon.org/other/haskell/Outputix/index.html>
 37. LTS Haskell 20.24 (ghc-9.2.7) - Stackage, <https://www.stackage.org/lts-20.24>
 38. Intervals and their relations : r/haskell - Reddit, https://www.reddit.com/r/haskell/comments/gdbs7n/intervals_and_their_relations/

39. @hackage > project-m36 > dependencies — Flora.pm,
<https://flora.pm/packages/@hackage/project-m36/1.2.6/dependencies>
40. Interval Container Library,
http://www.joachim-faulhaber.de/boost_icl/doc/libs/icl/doc/html/boost_icl/examples/interval.html
41. Interval - Boost,
https://www.boost.org/doc/libs/latest/libs/icl/doc/html/boost_icl/examples/interval.html
42. Chapter 1. Boost.Icl, <https://www.boost.org/libs/icl>
43. Always use [closed, open) intervals. A programmer's perspective - fhur.me,
<https://fhur.me/posts/always-use-closed-open-intervals>
44. IntervalSets.jl - Julia Packages, <https://juliapackages.com/p/intervalsets>
45. Home · IntervalSets.jl, <https://juliamath.github.io/IntervalSets.jl/>
46. Parser support for open/closed interval notation? - General Usage - Julia Discourse,
<https://discourse.julialang.org/t/parser-support-for-open-closed-interval-notation/43781>
47. Computer Arithmetic and Validity: Theory, Implementation, and Applications 9783110301793, 9783110301731 - DOKUMEN.PUB,
<https://dokumen.pub/computer-arithmetic-and-validity-theory-implementation-and-applications-9783110301793-9783110301731.html>
48. (PDF) Interval Arithmetic with Containment Sets - ResearchGate,
https://www.researchgate.net/publication/220261171_Interval_Arithmetic_with_Containment_Sets
49. A framework to test interval arithmetic libraries and their IEEE 1788-2015 compliance - arXiv, <https://arxiv.org/pdf/2307.06953>
50. Show HN: I made a calculator that works over disjoint sets of intervals | Hacker News, <https://news.ycombinator.com/item?id=47812341>
51. An Interval Arithmetic for Robust Error Estimation - arXiv,
<https://arxiv.org/pdf/2107.05784>
52. (PDF) Practical implementation of Interval Arithmetic - ResearchGate,
https://www.researchgate.net/publication/379413007_Practical_implementation_of_Interval_Arithmetic
53. A Standards compliant Interval Arithmetic Library - Nextjournal,
<https://nextjournal.com/krish8484/a-standards-compliant-interval-arithmetic-library>
54. Documentation: 18: 8.17. Range Types - PostgreSQL,
<https://www.postgresql.org/docs/current/rangetypes.html>
55. PostgreSQL - Jorge Israel Peña, <https://jip.dev/notes/postgresql/>
56. Data Types | openGauss documentation,
<https://docs.opengauss.org/en/docs/5.0.0/docs/BriefTutorial/data-types.html>
57. Documentation: 18: CREATE TYPE - PostgreSQL,
<https://www.postgresql.org/docs/current/sql-createtype.html>
58. Update lower/upper bound of range type - postgresql - Stack Overflow,
<https://stackoverflow.com/questions/18145852/update-lower-upper-bound-of-range-type>

[nge-type](#)

59. How exactly does inclusion in postgresql ranges work? - Stack Overflow,
<https://stackoverflow.com/questions/77221647/how-exactly-does-inclusion-in-postgresql-ranges-work>
60. PostgreSQL 16.4 Documentation - Docs,
<https://docs.jade.fyi/postgres/postgres-16.html>